

NEXA v1

Edge Cases, Failure Modes & Risk Register

Living engineering document • Use beside the architecture specification

Purpose: turn unexpected scenarios into explicit requirements, tests and recovery strategies.

1. How to Use This Document

This document supplements NEXA v1 Architecture Specification. It is not a feature wish-list. Every case should lead to one or more of: an invariant, an API rule, a recovery strategy, an automated test, or a documented product decision.

For each new feature:

1. Define happy path
2. Enumerate edge cases
3. Define failure behavior
4. Define authorization/privacy rules
5. Define consistency expectation
6. Add automated tests
7. Record unresolved decisions in ADRs

NOTE Do not attempt to solve every theoretical case before MVP. Use Priority P0/P1/P2/P3 to stage implementation.

2. Risk Rating

Priority	Meaning	Action
P0 — Critical	Security, tenant isolation, data corruption, auth bypass, message duplication/loss	Must be designed before implementation; must have automated coverage.
P1 — High	Important correctness/reliability issue likely in normal operation	Resolve before production MVP.
P2 — Medium	Scalability, UX degradation, operational inconvenience	Plan and implement before scale or when feature is hardened.
P3 — Future	Edge case unlikely to matter in v1	Document so it is not forgotten; do not block v1.

3. Global System Invariants

- ☐ Every tenant-owned resource is scoped to an Organization.
- ☐ Authentication success does not automatically imply organization authorization.
- ☐ Client-supplied organization_id must not override server-derived membership context.
- ☐ No raw password or raw refresh token is stored.
- ☐ Business mutations happen through application/domain services, not directly from controllers.
- ☐ Cross-module side effects use stable contracts/domain events where practical.
- ☐ Client retries must not create unintended duplicate durable operations.
- ☐ All externally visible IDs should be non-sequential identifiers such as UUID/UUIDv7.
- ☐ Critical data is persisted in PostgreSQL; Redis is not the system of record.
- ☐ Every critical authorization rule has an automated test.

4. Identity & Authentication Cases

4.1 Duplicate registration

Scenario:

User submits same email twice or two requests race.

Expected:

- unique DB constraint
- deterministic 409 response
- no partially created account

4.2 Login brute force

Risk: repeated password attempts.

Mitigation:

- rate limit
- audit suspicious attempts
- avoid revealing whether email exists

4.3 Stale access token after membership removal

Scenario:

10:00 login

10:05 admin removes member

10:06 access token is still cryptographically valid.

Rule:

JWT valid != organization access valid.

Membership/organization authorization must still be enforced.

4.4 Role change while token remains valid

Scenario:

MEMBER -> ADMIN -> MEMBER while old access token exists.

Decision:

Do not put full permission set into the JWT. Resolve effective organization permissions server-side or use a short-lived auth context that can be invalidated.

4.5 Refresh token replay

Risk:

A stolen refresh token is reused after rotation/revocation.

Recommended:

- hash refresh tokens at rest
- rotate on refresh
- revoke token family on reuse detection
- record session/device metadata

4.6 Account deletion/deactivation

Never treat account deletion as a simple SQL DELETE.

Decide separately for:

User identity

Posts

Comments

Messages

Reactions

Files

Audit logs

Organization membership

NOTE A common product outcome is: account becomes deactivated/anonymized while historical business records remain where legally or operationally required.

5. Multi-Tenancy & Authorization Cases

5.1 Cross-tenant object access

User belongs to Company B. Request references a post created in Company A.

Required:

`query(resource_id + organization_id)`

NOT `query(resource_id)` only.

5.2 Tenant enumeration

If resource IDs are predictable, users may probe neighboring IDs.

Mitigation:

UUID/UUIDv7 + authorization checks + careful error policy.

5.3 Multiple organization memberships

User belongs to Company A and Company B.

UI must have explicit active organization context.

API must derive/validate the active organization from authenticated membership.

5.4 Admin is not God mode

Admin may manage members and moderate content but should not automatically gain access to private DMs unless product policy explicitly defines such access.

5.5 Ownership vs permission

Post mutation:

owner -> allowed

moderator with `post.moderate` -> allowed

other member -> 403

non-member -> tenant authorization failure

6. Organization & Membership Cases

Case	Expected behavior	Priority
Invite already-active member	Return conflict or idempotent success; never create duplicate membership.	P1
Invite token expires	Reject; allow re-invite.	P1
Invite token reused	Reject after first successful acceptance.	P1
Admin removes self as last owner	Reject unless ownership is transferred.	P0
Department deleted	Define whether members move to no department or a fallback department.	P1
Member suspended	Block new mutations; define whether read access remains.	P1
User leaves organization	Remove membership but preserve permitted historical records.	P1

7. Social / Post Cases

7.1 Concurrent post update and delete

User sends UPDATE and DELETE nearly simultaneously.

Rule:

- define last-write / deletion precedence
- use updated_at/version check when needed
- mutation authorization occurs at mutation time

7.2 Like race condition

Same user sends LIKE twice concurrently.

Protection:

UNIQUE(user_id, post_id)

API should be idempotent where possible.

7.3 Delete post with comments/reactions

Recommended v1 direction:

soft-delete the post; hide it from normal feeds; retain enough metadata for audit and integrity. Define behavior for comments, reactions, attachments explicitly.

7.4 Nested comments

Cases:

- reply to deleted parent
- deeply nested tree
- comment owner deletes own comment
- moderator deletes comment

Recommendation:

keep nesting depth bounded in UI and preserve parent relationship in data model.

7.5 Mention deleted/inactive user

Mention parser should resolve identity at creation time. Historical content can retain a stable actor reference even after deactivation.

8. Feed & Pagination Cases

8.1 Cursor stability

Offset pagination can duplicate/skip items when new posts arrive.

Use cursor pagination based on a stable ordering key such as (created_at, id).

8.2 Post arrives while user is scrolling

Do not force-scroll the feed. Show a non-blocking “New posts” indicator and merge when user chooses.

8.3 Deleted item inside an existing cursor window

Feed must tolerate an item disappearing between requests. Cursor should remain valid; query simply returns the next eligible records.

8.4 Visibility changes

If a post changes from ORGANIZATION to DEPARTMENT or CHANNEL scope, future feed queries must enforce the new visibility immediately.

9. Chat: Delivery, Ordering & Reliability

9.1 Duplicate send on retry

Client sends message; network drops; client retries.

Use:

client_message_id / idempotency key

Server stores uniqueness per sender/conversation as appropriate.

9.2 Message ordering

Never trust client timestamps to define canonical ordering.

Use server-generated sequence/timestamp/monotonic ordering fields as needed.

9.3 Persisted but not acknowledged

Server persists message but websocket response is lost.

On reconnect, client syncs history and reconciles pending local messages by client_message_id.

9.4 WebSocket disconnect/reconnect

Disconnect flow:

1. mark connection gone
2. reconnect with auth
3. resubscribe to authorized conversations
4. sync missed messages/events
5. refresh presence
6. reconcile read state

9.5 Multiple tabs/devices

Presence should represent active connections, not a boolean per user.

A laptop closing must not set the user offline if their phone is still connected.

9.6 Read receipt conflict

Device A read message 200; Device B reports message 180.

Use monotonic advancement: last_read_message_id/sequence should move forward, not backward.

9.7 Message deletion

Prefer soft deletion with a tombstone such as “Message deleted” when history integrity matters.

Define whether attachments are retained/deleted and how audit behaves.

10. Chat Permission & Privacy Cases

Case	Decision to document	Priority
User removed from conversation	Cannot send new messages; historical visibility depends on conversation policy.	P0

User removed from organization	Revoke organization-scoped chat access; preserve permitted history.	P0
Private DM moderation	Admin cannot access by default unless explicit policy/legal requirement exists.	P0
Blocked user	Define whether direct messaging is blocked, and whether existing conversation remains visible.	P1
Channel archived	No new messages; read history remains available to permitted members.	P1
User joins channel late	Can they read full history or only from join time? Choose one policy.	P1

11. Presence Cases

Presence model:

User

- └─ Connection #1
- └─ Connection #2
- └─ Connection #3

Derived state:

0 connections → OFFLINE

>=1 active connections → ONLINE

TTL/heartbeat determines stale connections.

11.1 Heartbeat failure

If no heartbeat within TTL, connection is considered stale. Redis key expires automatically where practical.

11.2 Network sleep/mobile background

Do not rely solely on clean disconnect events. TTL is the recovery mechanism.

12. Notification & Event Cases

12.1 Event committed, notification failed

CommentCreated persists even if notification delivery fails.

Notification is an eventual-consistency side effect, not the transaction's source of truth.

12.2 Duplicate events

Consumers must be idempotent.

Store event IDs or processing keys where necessary.

12.3 Notification storm

Example:

1,000 users like one post.

Do not blindly create 1,000 visually identical notifications for the recipient.

Consider aggregation/coalescing policies later in v1/v2.

12.4 Actor is deleted

Notification should resolve gracefully, e.g. “A former member interacted with your post”, while preserving entity integrity.

13. File Upload & Storage Cases

Case	Expected behavior	Priority
Fake MIME type	Validate actual content where feasible; never trust client MIME alone.	P0
Executable/malicious upload	Reject dangerous types; define allowlist.	P0
Oversized file	Reject before expensive processing.	P1
Storage success / DB failure	Orphan cleanup job or transactional outbox-like workflow.	P1
DB success / storage failure	Do not expose broken file; mark upload failed.	P1
Same file referenced by multiple records	Do not physically delete while references remain.	P1
Broken CDN/object URL	UI must show fallback; metadata remains intact.	P2

14. Search & Privacy Cases

Search must be authorization-aware.

User may search only entities they are already permitted to see.

Effective search scope:

organization membership

+ resource visibility

+ channel/department membership

+ moderation/privacy rules

14.1 Search index lag

If v2 uses a separate index, newly created content may not appear instantly.

Product should tolerate eventual consistency.

15. Database & Concurrency Cases

15.1 Transaction boundaries

Use a DB transaction when multiple writes must succeed/fail together.

Do not wrap unrelated external side effects such as email/WebSocket delivery in a long database transaction.

15.2 Optimistic concurrency

For editable resources where lost updates matter, consider version or updated_at checks.

Example:

```
UPDATE posts
```

```
SET content = ...
```

```
WHERE id = ... AND version = expectedVersion
```

15.3 Deadlocks/timeouts

Operations touching multiple rows/tables should follow stable lock ordering.
Retry only safe/idempotent transactions.

15.4 Read-after-write expectations

After creating a post/message, the creator should immediately see the created entity.
Prefer the write response as the canonical object rather than waiting for a secondary read path.

16. Redis Failure & Degraded Mode

Capability	If Redis is down
Cache	Bypass cache and read PostgreSQL where safe.
Presence	Show unknown/stale/offline depending on policy; do not corrupt primary data.
Rate limiting	Use fallback limiter or temporarily fail closed for sensitive endpoints.
WebSocket Pub/Sub	Single-instance may still work; multi-instance fanout is degraded until Redis recovers.

17. API Abuse & Security Cases

Authentication endpoints

- brute force
- credential stuffing
- enumeration

Social

- spam posts/comments
- reaction abuse

Chat

- message flood
- connection flood

Upload

- malware
- storage exhaustion

Admin

- privilege escalation
- Apply endpoint-specific rate limits.
- Use request IDs and structured logs for investigation.
- Validate payload size before parsing/processing expensive content.
- Use least-privilege permissions.
- Keep security-sensitive operations auditable.

18. Privacy & Workplace Policy Decisions

NOTE These are product policy questions, not purely technical questions. Record the final decision in an ADR and expose the policy in the admin documentation.

- ☐ Can admins read private DMs?
- ☐ Can a user delete their own message permanently?
- ☐ How long are deleted messages retained?
- ☐ How long are audit logs retained?
- ☐ What happens to content created by an employee who leaves?
- ☐ Can an organization export all tenant data?
- ☐ Can an organization permanently delete its tenant?
- ☐ Can users block or mute coworkers?
- ☐ Can announcement posts be edited after publishing?

19. Moderation Cases

Post lifecycle:

ACTIVE → REPORTED → HIDDEN → REVIEWED → RESTORED/DELETED

Keep moderation actions auditable.

Separate user ownership from moderator authority.

19.1 Abuse report race

Many users report the same content simultaneously.

Use unique/aggregated reporting rules and avoid duplicate moderation jobs.

20. Offline / Mobile Reserve Cases

Reserved architecture for future React Native:

Offline message draft

- local queue
- network restored
- idempotent sync
- server acknowledgement
- reconcile local state

NOTE Do not implement full offline synchronization in v1 unless product validation makes it necessary. Preserve client_message_id/idempotency semantics now so future offline support remains possible.

21. Failure Matrix

Failure	User-visible behavior	Data risk	Recovery
PostgreSQL unavailable	503 / retry message	High	Restore DB; no silent writes.
Redis unavailable	Degraded realtime/cache/presence	Medium	Bypass optional paths; restore Redis.
Object storage unavailable	Upload fails gracefully	Medium	Retry upload; mark failed state.
WebSocket unavailable	Realtime disabled; polling/reconnect indicator	Low/Medium	REST history remains usable.
Notification worker unavailable	Actions succeed; notifications delayed	Low	Retry pending events.
Search unavailable	Show search unavailable	Low	Use primary features;

			restore index.
Email provider unavailable	Email delayed/failed	Low	Retry with queue/provider recovery.

22. Required Automated Test Suites

22.1 Security tests

- ☐ Cross-tenant GET returns denied/not found per policy.
- ☐ Cross-tenant UPDATE/DELETE is denied.
- ☐ Non-member cannot access organization resources.
- ☐ Admin-only endpoint rejects normal member.
- ☐ Owner-only post update/delete is enforced.
- ☐ Expired/revoked refresh token is rejected.
- ☐ Password hash never appears in API response.

22.2 Concurrency tests

- ☐ Duplicate registration race results in one account.
- ☐ Duplicate like requests create one reaction.
- ☐ Duplicate message retry creates one message.
- ☐ Read receipt never moves backward.
- ☐ Concurrent post update/delete follows documented policy.

22.3 Realtime tests

- ☐ WebSocket reconnect restores authorization and subscriptions.
- ☐ Missed messages are synchronized after reconnect.
- ☐ Multiple devices produce correct presence.
- ☐ Message event is not delivered twice to the same effective client state.

22.4 Data integrity tests

- ☐ Deleting/deactivating user follows documented retention rules.
- ☐ Deleting post follows comment/reaction/attachment policy.
- ☐ Orphan file records can be detected.
- ☐ Invalid foreign organization references are rejected.

23. Risk Register

Risk	Impact	Likelihood	Priority	Mitigation
Cross-tenant data leak	Critical	Medium	P0	Mandatory organization scope + authorization tests.
Privilege escalation	Critical	Medium	P0	RBAC + resource ownership checks + negative tests.
Duplicate chat messages	High	High	P0	client_message_id/idempotency.
Stale membership access	High	Medium	P0	server-side membership authorization.
Malicious file	Critical	Medium	P0	allowlist + size/content checks +

upload				isolated storage.
Notification duplication	Medium	High	P1	idempotent event consumers.
Feed query slowdown	High	Medium	P1	indexes + cursor pagination + measured optimization.
Redis outage degrades realtime	Medium	Medium	P1	graceful degradation + health monitoring.
Search privacy leak	Critical	Low/Medium	P0	authorization-aware search scope.
Orphan files	Medium	Medium	P1	cleanup/reconciliation job.
Lost concurrent edits	Medium	Medium	P1	version/updated_at optimistic concurrency where needed.
Presence false-offline	Low/Medium	High	P1	connection tracking + TTL.
Admin privacy overreach	Critical	Medium	P0	explicit policy; deny private DM access by default.
Overengineering delays MVP	Medium	High	P1	modular monolith; P0/P1 gates; defer v2 systems.

24. Open Product Decisions

Decision	Why it matters	Target phase
Private DM admin access policy	Workplace privacy and trust	Before production
Message retention period	Storage, privacy, compliance	Before production
Tenant deletion/export	Enterprise operations	v1/v1.1
Channel history visibility for new members	UX/privacy	v1
User blocking rules	Social/HR dynamics	v1.1
Announcement editing window	Content integrity	v1
Attachment size/types	Security and cost	v1
Notification aggregation policy	Noise at scale	v1.1
Search implementation threshold	Scaling trigger for external search	v2
Offline sync	Mobile UX complexity	v2+

25. Engineering Checklist Before Production

- ☐ Tenant isolation tests pass.
- ☐ Authorization negative tests pass.
- ☐ Refresh token rotation/revocation works.
- ☐ Rate limits are configured for auth and abuse-prone endpoints.
- ☐ Upload validation and storage permissions are configured.
- ☐ WebSocket reconnect and message reconciliation are tested.
- ☐ Critical DB indexes are verified using realistic queries.
- ☐ Health/readiness checks reflect actual dependency health.
- ☐ Logs contain request_id and useful event identifiers.
- ☐ Backups/restore procedure for PostgreSQL is documented.
- ☐ Incident response/runbook exists for auth, data leak and outage scenarios.
- ☐ Privacy policy decisions are written and visible to administrators where appropriate.

26. Rule for Future Features

When adding a new feature, do not only ask:
“What endpoints do we need?”

Also ask:
“What can go wrong?”

Minimum design template:

Feature

- Happy path
- Actors
- Permissions
- Tenant scope
- State transitions
- Failure modes
- Idempotency
- Consistency
- Recovery
- Audit
- Tests
- Open decisions

27. Final Principle

NEXA should feel simple to users even when the engineering underneath is careful. The product should hide complexity, while the codebase makes security, tenant isolation, reliability, privacy and recovery explicit.

Simple UX

↓

Strong domain rules

↓

Predictable APIs

↓

Recoverable failures

↓

Trustworthy platform